

SPECTRAL ENVELOPE MODELING

Converting PSD to FIR weights via Linear Predictive Coding (LPC)

The Conceptual Bridge

In many DSP applications, we don't need the fine-grained noise of a spectrum, but rather the **envelope** that defines its shape. Converting a Power Spectral Density (PSD) to FIR weights (LPC coefficients) is the standard engineering method to achieve this.

The relationship is governed by the **Wiener-Khinchin Theorem**, which states that the PSD and the Autocorrelation function are a Fourier Transform pair.

$$PSD(f) \xrightarrow{\text{IFFT}} \text{Autocorrelation}(R_{xx}) \xrightarrow{\text{Yule-Walker}} \text{LPC Coefficients}(a)$$

Step-by-Step implementation Guide

STEP 1

Target Definition

First, we define a desired magnitude response. In a real scenario, this would be your measured PSD. Here, we create a synthetic one with two resonances.

```
fs = 16000; % Sampling Frequency
f = linspace(0, fs/2, 513);

% Create a synthetic PSD (linear scale)
% Two peaks: 1200Hz and 3500Hz
psd = 1./(1 + 0.05*(f-1200).^2) + 0.5./(1 + 0.01*(f-3500).^2) + 0.01;

plot(f, 10*log10(psd)); grid on;
```

```
title('Input Power Spectral Density'); xlabel('Hz'); ylabel('dB');
```

STEP 2

The Wiener-Khinchin Transformation

To solve for filter weights, we must move to the **Lag Domain** (Autocorrelation). We mirror the PSD to make it double-sided, ensuring the result of the IFFT is purely real.

```
% Create double-sided PSD for IFFT
psd_double = [psd, fliplr(psd(2:end-1))];

% Compute Autocorrelation via IFFT
rxx = real(ifft(psd_double));

% Keep only the lags needed for our filter order
p = 24; % Filter Order
r_lags = rxx(1:p+1);
```

STEP 3

Solving the Yule-Walker Equations

The Yule-Walker equations relate these lags to the optimal linear predictors. MATLAB's `levinson` function handles the recursion efficiently.

```
% Solve for LPC coefficients
% 'a' are the FIR weights of the analysis (whitening) filter
% 'e' is the prediction error power (gain factor)
[a, e] = levinson(r_lags, p);

fprintf('Generated %d FIR weights.\n', length(a));
```

Verification & Envelope Extraction

Finally, we verify that our FIR filter's magnitude response matches the original PSD's envelope.

```
% Compute frequency response of the LPC model
[h, w] = freqz(sqrt(e), a, 513, fs);

hold on;
plot(w, 20*log10(abs(h)), 'r', 'LineWidth', 2);
legend('Original PSD', 'LPC FIR Model');
title('Spectral Envelope Fitting');
```

Summary of the Results

- **FIR Weights:** The vector a acts as an FIR filter. When applied to the signal, it "whitens" the spectrum (flattens the peaks).
- **The Envelope:** The inverse, $1/A(z)$, represents the smoothed spectral envelope.
- **Efficiency:** You have effectively compressed a 513-point spectral description into just **25 coefficients**.